

Communication Reduction via Semantic-Based Encoding in DMPC Using LSTMs

Torben Schiz¹, Pedro H. J. Nardelli², and Henrik Ebel¹

¹Dynamics & Control, Dept. of Mechanical Engineering, LUT University, Lappeenranta, Finland

²Cyber-Physical Systems, Dept. of Electrical Engineering, LUT University, Lappeenranta, Finland

Abstract

The communication demands of distributed model predictive control (DMPC) can overwhelm even advanced wireless communication technologies as agents must exchange a significant amount of information at least once per time step. To semantically reduce communication demands, this work employs encoder-decoder networks built around long-short term memory (LSTM) cells in a distributed optimization algorithm. Agents publish a reduced representation of a message and receivers reconstruct the original message upon reception. In tests with reduced communication using formations of mobile robots, trained networks retain satisfactory performance and work reliably under conditions overwhelming full communication. As the results show, the usage of LSTMs either allows unprecedented reconstruction accuracy or the usage of different prediction-horizon lengths without the necessity to retrain.

Keywords: Distributed MPC, communication reduction, LSTM, distributed control, semantic communication, multi-agent systems, robotics

1 Introduction

Networked control has long been heralded to be more robust, adaptable, and scalable than centralized control methodologies and better performing than decoupled ones. A timely application area are especially groups of collaborating robots, as ground-based and air-based robots have made headlines and created disruption in sectors like logistics, security, and defense. Still, in robotics, the vast majority of present-day real-world applications, even if advanced automation is present, typically do not include autonomous swarm-like collaborative behavior and self-reliant multi-robot collaboration. Apart from application-end engineering challenges, some of the reasons for this may also be found in the demands of control methods proposed to achieve such behavior. In particular, and in sole focus of this work, model predictive control (MPC), which has become widely popular in industrial applications [1], holds big promises in its distributed variants also for networked and collaborative control applications. Distributed variants can also benefit from MPC's general advantages like an easy-to-define and easy-to-alter control goal, inclusion of (po-

tentially collaborative) constraints, direct consideration of nonlinear dynamics, and naturally optimized performance. Many such distributed model predictive control (DMPC) schemes and algorithms have been proposed, tailored to different control-problem setups and often with the aim to prove theoretical guarantees for the closed-loop behavior [2–7]. In DMPC, each agent (e.g., each mobile robot) makes its own decisions by solving its own optimal control problem, removing any single point of failure represented by a centralized decision maker. However, each agent's OCP considers information from those other agents that impact the optimality of its own decisions through couplings in the control goals (cost function), the constraints, and/or the dynamics. Such agents are called neighbors subsequently, and, in mobile robotics, the information exchange typically needs to happen explicitly via wireless communication.

In stark contrast to MPC, to the best knowledge of the authors, real-world applications of DMPC are presently lacking, especially in robotics, where the latter is in focus of this article due to the many worthwhile applications it presents to networked control. One reason might be, as for the application of MPC, a lack of computation power to solve the underlying optimal control problems (OCPs) in a timely-enough manner, especially considering that the OCPs then typically need to be solved on on-board computers tracking behind personal and industrial computers in performance. Indeed, earlier works see limited computation resources as an inhibitor for DMPC applications [8]. However, the progress in mobile compute performance has been swift, and a recent work finds that, actually, “communication is the bottleneck” [9] in real-world DMPC applications in robotics where agent-to-agent communication happens wirelessly and computation takes place on board of each robot. This may seem surprising as also communication technology has experienced progress. However, common communication technologies are typically designed to be as application agnostic as possible, aiming at the reliable and timely delivery of messages independently of their relevance to the task. Semantic communications, including task-oriented and goal-oriented approaches, instead assign value to information according to its significance for the decision or actuation process performed at the receiver [10, 11]. More generally, semantic communications incorporate the application objective into the communication process, enabling task-relevant information to be represented and transmit-

ted using fewer communication resources. This gives rise to semantic encoding and decoding mechanisms that can reduce network traffic and facilitate timely delivery, although these gains may require increased computational complexity at the transmitter and receiver [12]. This perspective is particularly suitable for networked control, where communication, computation, and control performance are intrinsically coupled [13], and where the value of information and requirements emanating from those for communication might be vastly different than for more common use-cases of present communication technology as, e.g., represented by typical usage patterns of smartphones or personal computers. In this context, semantic communications can reduce the communication burden by prioritizing compact, task-relevant information, thereby improving scalability and enabling informative messages to be delivered within the timing constraints of the control system [14, 15].

In this context, this article shows that the (communication-related) performance of DMPC can be vastly improved by a semantics-based encoding of the communication information on application level, without touching lower-level elements of the communication stack, preserving straight-forward applicability using standard communication technologies and protocols. To that end, the article proposes a machine-learning-based reduction of to-be-communicated data to a latent space at the transmitting agents prior to communication paired with a decoder at the receiving agents, which reconstructs the original message. Novelty, one of the proposed approaches almost perfectly recovers the closed-loop performance of lossless, uncompressed communication while being much more robust to real-world imperfections of communication. Further, we propose approaches that need not be retrained every time a different prediction horizon is picked, trading some of the performance for generality. These approaches are based on encoder-decoder recurrent neural networks [16], which can be trained for arbitrary sequence lengths in a single training run.

Commonly in robotics, there is a simulation-to-reality gap emanating from the complexity of the real system dynamics. However, there exist many successful experiments involving simple mobile robots, and models can capture the underlying physics well, making it possible to switch from simulation to experiment with relative ease in certain applications [17, 18]. However, in DMPC for mobile robots, there is a significant, additional sim-to-real gap introduced by communication, as the communication is often simplified in experiments by solving the OCP on external compute units, for example in [18–23], thus, bypassing wireless inter-agent communication.

In control literature, to the best of the authors’ knowledge, only few works attempt to reduce the communication demands by reducing the message sizes in DMPC, e.g., for collaborating mobile robots, and research streams are largely separated from research on communications technology [24]. A non-data-driven approach to reducing the communication in DMPC in robotics is employed in [25] and [26], whereas [27] and [28] reduce the communication using data-driven methods. Of the data-driven approaches,

[27] communicates the parameters of a neural network that is re-trained in every step prior to communication leading to prohibitively long computation times. The inter-agent communication is reduced using an autoencoder in [28], where, prior to communication, the to-communicate information is passed through the (semantic) encoder and after reception through the (semantic) decoder part, reconstructing the original message. With this approach, simulated robot formations successfully completed tasks under communication conditions where the uncompressed communication failed. No other neural network architectures were tested and compared to the autoencoder. Additionally, as a standard autoencoder is trained for fixed input and output layer sizes, the approach from [28] requires retraining whenever the horizon lengths is changed. In other words, for each horizon length a separate autoencoder is needed, complicating controller tuning as the prediction horizon is often seen as a key tuning parameter in (D)MPC. Thus, the state of the art leaves two key research questions. Firstly, the question arises whether other neural-network architectures can be trained to achieve better results for a fixed prediction horizon length. Secondly, one may ask how architectures that generalize to a variable horizon length can be furnished, removing the need for explicit retraining whenever the prediction horizon length is changed. Both questions are attended in this work as follows. Section 2 introduces a general DMPC optimal control problem and adapts it subsequently for formations of nonholonomic robot formations for asymptotically stabilizing setpoint tasks. The models used for communication reduction are introduced in Section 3, followed by a description of the training process in Section 4. The trained models are first validated in simulation to compare their respective performances when used in the optimization algorithm under ideal communication conditions (Section 5) before testing the communication reduction models under realistic conditions on embedded hardware (Section 6). Finally, Section 7 summarizes the findings.

2 Problem Setting

This section first provides a general distributed MPC setup in Section 2.1 highlighting that the proposed communication reduction methods can be applied to a wide class of cooperative control problems. Then, Section 2.2 shows how DMPC can be applied to formation control of mobile robots as the exemplary application of used in this work.

2.1 General Setup

This work considers a quite optimal control problem furnished as follows. A cost function $J(\mathbf{z}(t), \mathbf{u}(\cdot|t)) = \sum_{k=0}^{H-1} \ell(\mathbf{z}(t+k|t), \mathbf{u}(t+k|t))$ couples $N \geq 2$ dynamically decoupled systems by defining a common goal. In the cost function, $\ell: \mathbb{R}^{n_z} \times \mathcal{U} \rightarrow \mathbb{R}$ is the so-called stage cost and the length of the prediction horizon is $H \in \mathbb{N}$. The states and inputs are given by $\mathbf{z} \in \mathbb{R}^{n_z}$ and $\mathbf{u} \in \mathbb{R}^{n_u}$, respectively. The inputs are subject to independent, per-



Figure 1: Photograph of differential-drive mobile robots as a common example for nonholonomic mobile robots

system constraints, constraining the concatenated inputs to the set $\mathcal{U} = \mathcal{U}_1 \times \dots \times \mathcal{U}_N$. The notation $(\cdot | t)$ marks a whole discrete-time trajectory along the prediction horizon as predicted at time step t , of which $(k | t)$ denotes the specific prediction for time step k where $k \in \mathbb{Z}_{t:t+H-1}$ with $\mathbb{Z}_{a:b} := \{a, \dots, b\}$. With this, the control task can be formulated in the form of the OCP

$$\underset{\mathbf{u}(\cdot | t)}{\text{minimize}} \quad J(\mathbf{z}(t), \mathbf{u}(\cdot | t)) \quad (1)$$

$$\text{subject to} \quad \mathbf{z}(t+k+1 | t) = \mathbf{f}(\mathbf{z}(t+k | t), \mathbf{u}(t+k | t)), \quad (2)$$

$$\mathbf{u}(t+k | t) \in \mathcal{U}, \quad k \in \mathbb{Z}_{0:H-1}, \quad (3)$$

$$\mathbf{z}(t | t) = \mathbf{z}(t), \quad (4)$$

which we aim to solve distributedly in the typical receding-horizon manner of MPC. Further, Eq. (2) is the appropriately discretized dynamics of the multi-agent system, where, in this article's case, the dynamics of the involved robots are decoupled, meaning that they can be simply concatenated.

2.2 Model Problem

In the following, nonholonomic mobile robots, as shown in Fig. 1, are used as test systems, although the methods developed in this article do not explicitly depend on the very concrete control task and are adjusted to it only via observation data during training. The continuous system dynamics of a single nonholonomic robot moving in a two-dimensional plane can be modeled using first-order kinematics as

$$\dot{\mathbf{z}}_i(t) = \begin{bmatrix} \dot{x}_i(t) \\ \dot{y}_i(t) \\ \dot{\theta}_i(t) \end{bmatrix} = \begin{bmatrix} v_i(t) \cos(\theta_i(t)) \\ v_i(t) \sin(\theta_i(t)) \\ \omega_i(t) \end{bmatrix} \quad (5)$$

with the state vector $\mathbf{z}_i(t) = [x_i(t) \ y_i(t) \ \theta_i(t)]^\top \in \mathbb{R}^{n_i}$, $n_i = 3$ and the inputs $\mathbf{u}_i(t) = [v_i(t) \ \omega_i(t)]^\top \in \mathbb{R}^{m_i}$, $m_i = 2$. The state vector consists of the position in the plane $[x_i(t) \ y_i(t)]^\top$ and the orientation $\theta_i(t)$ measured relative to the x axis. The input is made up of the linear velocity $v_i(t)$ and the angular velocity $\omega_i(t)$. The nonholonomic differential-drive robot has two parallel wheels as well as a castor wheel or ball. The implicitly fulfilled Pfaffian constraint $[\sin \theta_i \ -\cos \theta_i \ 0] \dot{\mathbf{z}}_i = 0$ on the kinematics prevents instantaneous lateral motion of the robot.

In the following, a brief description of the concrete exemplary formation-control task for nonholonomic robots as

solved in this work's numerical experiments is given. The task has previously been studied in [20], where a more detailed derivation can be found. The overall system dynamics of the decoupled system results from concatenating the states and inputs of the individual robots to

$$\dot{\mathbf{z}}(t) = [\dot{\mathbf{z}}_1^\top(t) \ \dots \ \dot{\mathbf{z}}_N^\top(t)]^\top, \quad \mathbf{z}(0) = \mathbf{z}_0. \quad (6)$$

The discretization in time with zero-order hold on the inputs and with sampling time Δt is denoted as $\mathbf{z}(t + \Delta t) = \mathbf{G}_d(\mathbf{z}(t), \mathbf{u}(t))$.

The output $\mathbf{y}^R \in \mathbb{R}^{3(N+1)}$, describing the robot formation, is expressed through the relation $\mathbf{y}^R(\mathbf{z}, \hat{\theta}_d) = \mathbf{C}^R(\hat{\theta}_d) \mathbf{z}$ where $\hat{\theta}_d$ is the desired orientation of the geometric center of the formation and the superscript $\bullet^R$ is used to describe quantities expressed in the auxiliary reference frame $\mathcal{K}_R$. The auxiliary reference frame $\mathcal{K}_R$ arises by rotation of the inertial frame of reference by $\hat{\theta}_d$. Here and in the following, $\bullet$ denotes a variable related to the geometric center of the formation. Mapping the concatenated state vector to the output via the matrix $\mathbf{C}^R \in \mathbb{R}^{3(N+1) \times 3N}$ adds the geometric center as the average of the poses of the robots in the formation, expresses the positions of each robot relative to said geometric center, and describes the positions in an auxiliary frame of reference $\mathcal{K}_R$. The orientation of each robot and the geometric center remain given in the inertial frame of reference. Describing the positions of the robots relative to the geometric center allows penalizing the absolute position of the geometric center and the relative positioning within the formation individually. To ensure that all robots and the geometric center can approach the desired setpoint along the x^R -axis, only such scenarios are considered where all robots in the formation have identical desired orientations. Adding all robots in the formation and the geometric center to the output introduces a redundancy to the formulation, which needs to be accounted for when picking a specific desired output

$$\mathbf{y}_d^R = [\hat{x}_d^R \ \hat{y}_d^R \ \hat{\theta}_d \ x_{\hat{z} \rightarrow 1,d} \ y_{\hat{z} \rightarrow 1,d} \ \theta_{1,d} \ \dots \ \dots \ x_{\hat{z} \rightarrow N,d} \ y_{\hat{z} \rightarrow N,d} \ \theta_{N,d}]^\top \in \mathbb{R}^{3(N+1)}, \quad (7)$$

with $\theta_{j,d} = \hat{\theta}_d$ and where the subscript $\hat{z} \rightarrow j, d$ denotes a value as being relative to the geometric center for all $j \in \mathcal{N}$ with $\mathcal{N}$ the number of agents in the formation. The cost function coupling the control systems and defining the common goal for each robot is chosen as

$$J(\mathbf{z}(t), \mathbf{u}(\cdot | t), \mathbf{y}_d^R) = \sum_{k=0}^{H-1} \ell(\mathbf{z}(t+k | t), \mathbf{u}(t+k | t), \mathbf{y}_d^R). \quad (8)$$

defined via the stage cost

$$\ell(\mathbf{z}, \mathbf{u}, \mathbf{y}_d^R) = \hat{\ell}(\mathbf{z}, \mathbf{u}, \mathbf{y}_d^R) + \sum_{j=1}^N (\ell_{j,\text{rel}}(\mathbf{z}, \mathbf{u}, \mathbf{y}_d^R) + \ell_{j,u}(\mathbf{u})), \quad (9)$$

composed of

$$\hat{\ell}(\cdot) := \hat{d}_1 (\hat{x}^R - \hat{x}_d^R)^4 + \hat{d}_2 (\hat{y}^R - \hat{y}_d^R)^2 \quad (10)$$

$$+ \hat{d}_3 (\hat{\theta}^R - \hat{\theta}_d^R)^4, \quad (11)$$

$$\hat{\ell}_{j,\text{rel}}(\cdot) := \hat{d}_{1,j} (\hat{x}_{\hat{z} \rightarrow j}^R - \hat{x}_{\hat{z} \rightarrow j,d}^R)^4 + \hat{d}_{2,j} (\hat{y}_{\hat{z} \rightarrow j}^R - \hat{y}_{\hat{z} \rightarrow j,d}^R)^2 \\ + \hat{d}_{3,j} (\hat{\theta}_j^R - \hat{\theta}_{j,d}^R)^4, \quad (12)$$

$$\ell_{j,u}(\cdot) := r_{1,j} v_j^4 + r_{2,j} \omega_j^4 \quad (13)$$

with the weights $\hat{d}_i, d_{i,j}, r_{m,j} \in \mathbb{R}_{>0}$, $i \in \mathbb{Z}_{1:3}$, $j \in \mathcal{N}$, $m \in \mathbb{Z}_{1:2}$. The weights are chosen for $j \in \mathcal{N}$ as $\hat{d}_1 = d_{1,j} = 1$, $\hat{d}_2 = d_{2,j} = 5$, $\hat{d}_3 = d_{3,j} = 0.1$, $r_{1,j} = 0.125$, and $r_{2,j} = 0.0125$ based on [29]. The stage cost uses both quadratic and quartic terms as described in [20] following [29] as even a single nonholonomic robot cannot be asymptotically stabilized to a full-state setpoint using merely quadratic terms when no terminal ingredients are used. Combining the previously introduced ingredients yields the distributed optimal control problem for robot $i \leq N$ at time t as

$$\underset{\mathbf{u}_i(\cdot|t)}{\text{minimize}} \quad J(\mathbf{z}(t), \mathbf{u}(\cdot|t), \mathbf{y}_d^R) \quad (14)$$

$$\text{subject to} \quad \mathbf{z}(t+k+1|t) =$$

$$\mathbf{G}_d(\mathbf{z}(t+k|t), \mathbf{u}(t+k|t)), \quad (15)$$

$$\mathbf{u}(t+k|t) \in \mathcal{U}, \quad k \in \mathbb{Z}_{0:H-1}, \quad (16)$$

$$\mathbf{z}(t|t) = \mathbf{z}(t). \quad (17)$$

Each robot optimizes only for its own control inputs $\mathbf{u}_i$ and not for the control inputs of the neighboring robots $j \in \mathcal{N} \setminus \{i\}$. Inputs are constrained using box constraints $\mathcal{U} = [\underline{v}, \bar{v}] \times [\underline{\omega}, \bar{\omega}] \subset \mathbb{R}^2$ with $\underline{v} \leq \bar{v}$ and $\underline{\omega} \leq \bar{\omega}$.

To solve the distributed OCP, the robots in the formation use the distributed optimization algorithm from [4], which has been adopted for mobile robots in [30]. At every time step $t \leq N_{\text{end}}$ where N_{end} is the number of time steps considered in the experiment, each robot conducts a line search to find a candidate input sequence optimizing only its own input, then exchanges this prediction with its neighboring systems and finally ensures the solution is convex-like for each iteration $p \leq \bar{p}$, where $\bar{p}$ is the number of iterations per time step. Due to the fixed iteration limit, solutions may be suboptimal. In the studied setup, each robot is each other robot's neighbor, resulting in a maximally challenging communication topology. At $p = \bar{p}$, each robot applies the first input of the found candidate input sequence. The techniques explored in this work reduce the inter-agent communication by encoding the candidate input sequence prior to publication and each encoded sequence is decoded once received.

3 Neural Network Architectures for Communication Reduction

To address the communication challenges in DMPC, in the following, five different neural-network architectures are

used to reduce the inter-agent communication in the distributed control algorithm from [4]. An overview over the architectures, which all have an LSTM-based encoder in common, is given in Table 1. Other than the shallow recurrent decoder (SHRED) [31], which uses a standard feed-forward neural network (FNN) as decoder, the networks use an LSTM not only in the encoder but also as part of the decoder. SHRED performed well in [31] for the reconstruction of high-dimensional states from sensor data motivating an attempt to use it in a novel application.

Using an encoder-decoder LSTM [16] architecture allows models to be trained and employed for multiple horizon lengths using a single training run. The use of different encoder-decoder architectures is motivated by limited time available for the encoding and decoding operations as well as restricted data transfer from encoder to decoder. The former limits the layer sizes and depth of the network, whereas the latter confines the amount of information from the encoder the decoder has available. It is uncertain whether communicating the cell state, the output, or a combination of the two can capture the most amount of information in the latent space. Thus, networks differ slightly depending on the quantity representing the reduced representation. Communicating a combination of cell state and output leads to the introduction of projection layers to reduce the dimension of the hidden state and network output in some networks. In Table 1, these layers are indicated by projected and FFL (feed-forward layer), respectively. The autoencoder from [28] serves as a baseline for reduced communication.

To reduce the inter-agent communication, the predicted input sequence along the prediction horizon $\mathbf{u}_i^{[p+1]}(\cdot|t)$ at iteration p and time step t of each agent $i \in \mathcal{N}$ is compressed into a latent space using a pre-trained encoder prior to the publication of the message. The reduced representation $\mathbf{r}_{c,i}^{[p+1]}(\cdot|t)$ is communicated and when another robot receives the message, the predicted input sequence is reconstructed using a decoder, as shown in Fig. 2. The same pre-trained model is used in all robots and each message is reduced and reconstructed individually, making the communication reduction independent of the formation size. Given the LSTM-input $\mathbf{u}_\tau$, one forward pass through an LSTM cell at index τ corresponds to

$$\mathbf{i}_\tau = \sigma(\mathbf{W}_{ii}\mathbf{u}_\tau + \mathbf{b}_{ii} + \mathbf{W}_{hi}\mathbf{h}_{\tau-1} + \mathbf{b}_{hi}) \quad (18)$$

$$\mathbf{f}_\tau = \sigma(\mathbf{W}_{if}\mathbf{u}_\tau + \mathbf{b}_{if} + \mathbf{W}_{hf}\mathbf{h}_{\tau-1} + \mathbf{b}_{hf}) \quad (19)$$

$$\mathbf{g}_\tau = \tanh(\mathbf{W}_{ig}\mathbf{u}_\tau + \mathbf{b}_{ig} + \mathbf{W}_{hg}\mathbf{h}_{\tau-1} + \mathbf{b}_{hg}) \quad (20)$$

$$\mathbf{o}_\tau = \sigma(\mathbf{W}_{io}\mathbf{u}_\tau + \mathbf{b}_{io} + \mathbf{W}_{ho}\mathbf{h}_{\tau-1} + \mathbf{b}_{ho}) \quad (21)$$

$$\mathbf{c}_\tau = \mathbf{f}_\tau \odot \mathbf{c}_{\tau-1} + \mathbf{i}_\tau \odot \mathbf{g}_\tau \quad (22)$$

$$\mathbf{h}_\tau = \mathbf{o}_\tau \odot \tanh(\mathbf{c}_\tau) \quad (23)$$

with the input gate $\mathbf{i}_\tau$, the forget gate $\mathbf{f}_\tau$, the cell gate $\mathbf{g}_\tau$ and the output gate $\mathbf{o}_\tau$. Further, $\mathbf{c}_\tau$ corresponds to the cell state, $\mathbf{h}_\tau$ to the hidden state, $\sigma(x) = 1/(1 + \exp(-x))$ is the sigmoid function, and $\odot$ the Hadamard product. The quantities $\mathbf{W}_{lm}$ and $\mathbf{b}_{lm}$ for $l \in \{i, h\}$ and $m \in \{i, f, g, o\}$ are the weights and biases, respectively. This basic LSTM architecture is modified slightly, depending on the neu-

Table 1: Overview over the considered models providing their name, encoder and decoder architectures as well as the quantity they communicate

Name	Encoder	Decoder	Communicate
SHRED [31]	LSTM	FFL	network output
Cell LSTM	LSTM	LSTM	cell state
LSTMP [32]	LSTM projected	LSTM projected	network output and cell state
LSTM+FFL	LSTM and FFL	LSTM and FFL	network output and cell state
LSTM+FFLH	LSTM	LSTM and FFL	network output

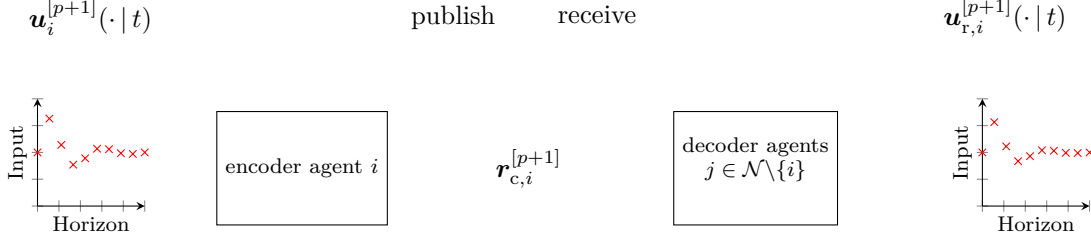


Figure 2: Semantic encoder-decoder structure: The encoded input sequence is published by robot i and received and decoded by the other robots in the formation $j \in \mathcal{N} \setminus \{i\}$.

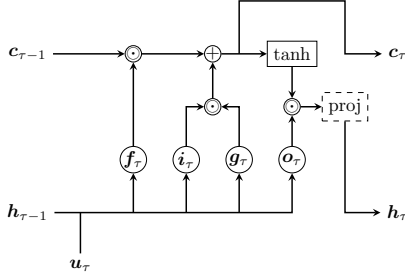


Figure 3: Sketch of an encoder LSTM cell with optional projection layer

ral network architecture used, e.g., by adding a projection layer. In the forward pass, for each robot i at time t and iteration p it holds that $\mathbf{u}_{\tau,i}^{\text{en}} = \mathbf{u}_i^{[p+1]}(t + k | t)$ for $(\tau, k) \in \{(1, 0), (2, 1), \dots, (H, H - 1)\}$. A cell of this LSTM encoder is displayed in Fig. 3. As the same network is used for each agent, the robot index i is omitted in the following for variables related to neural networks. SHRED uses a standard LSTM encoder combined with a feed-forward decoder. The use of an FNN as decoder requires retraining when the length of the prediction horizon is changed. The output of the encoder, i.e., $\mathbf{r}_c = \mathbf{h}_H$, is communicated and used as input by the decoder. A sketch of this architecture is presented in Fig. 4a.

In the variants using LSTMs also in the decoder, the input predictions are reconstructed in reverse order, such that $\mathbf{u}_r(t + H - 1 - l | t) = \mathbf{u}_{\tau}^{\text{de}}$ for $(\tau, l) \in \{(1, 0), (2, 1), \dots, (H, H - 1)\}$. The first input $\mathbf{u}_{\tau=0}^{\text{de}}$ is not used in the reconstruction of the control input vector but only used as an LSTM input.

The variant denoted as cell LSTM communicates only the cell state of the encoder $\mathbf{r}_c = \mathbf{c}_H^{\text{en}}$. The decoder sets the initial input and hidden state to zero, i.e., $\mathbf{h}_0^{\text{de}} = \mathbf{0}$ and $\mathbf{u}_0^{\text{de}} = \mathbf{0}$, and uses the cell state it received as the initial cell

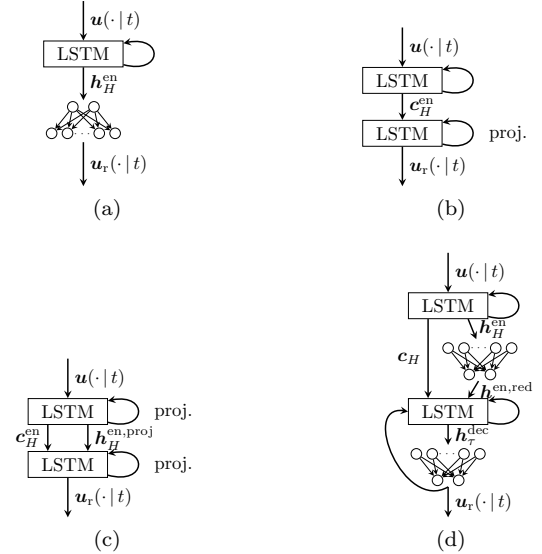


Figure 4: Overview over different LSTM architectures: (a) SHRED, (b) Cell LSTM, (c) LSTMP, (d) LSTM+FFL

state meaning that $\mathbf{c}_H^{\text{de}} = \mathbf{r}_c$. An overview of this architecture is given in Fig. 4b.

The LSTMP architecture, based on LSTM projected [32] and depicted in Fig. 4c, communicates the cell state and projected hidden state, i.e., $\mathbf{r}_c = [\mathbf{c}_H^{\text{en}T} \quad \mathbf{h}_{\text{proj},H}^{\text{en}T}]^T$. In this model, the output of the LSTM is projected in both the encoder and the decoder to reduce its dimension to the dimension of a velocity pair using a projection layer of the form $\mathbf{h}_{\text{proj},\tau} = \mathbf{W}_{\text{proj}} \mathbf{h}_{\tau}$ where $\mathbf{W}_{\text{proj}}$ is a learnable weight matrix.

The projected hidden state is used as the input to the decoder, i.e., $\mathbf{u}_0^{\text{de}} = \mathbf{h}_{\text{proj},H}^{\text{en}}$, and the last cell state of the encoder is used as the initial first cell state of the decoder meaning that $\mathbf{c}_0^{\text{de}} = \mathbf{c}_H^{\text{en}}$. The initial hidden states are set to zero.

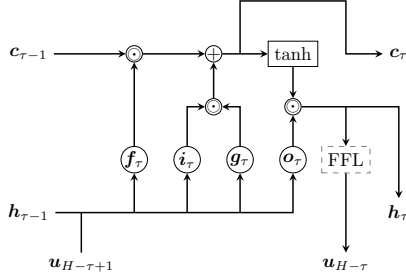


Figure 5: Sketch of a decoder LSTM with an FFL. The FFL is only applied to the output of the last layer at $\tau = H$. Note that the prediction sequence is recreated in reverse order from H to 0 in the decoder.

In the LSTM+FFL variant, sketched in Fig. 4d, instead of using a projection in each pass of the LSTM, only the output of the encoder is projected to a two-dimensional representation using a single-layer FNN, i.e.,

$$\mathbf{h}_{\text{FFL},H} = \mathbf{W}_{\text{FFL}}a(\mathbf{h}_H) + \mathbf{b}_{\text{FFL}} \quad (24)$$

with the weight matrix $\mathbf{W}_{\text{FFL}}$, the bias vector $\mathbf{b}_{\text{FFL}}$, and the activation function a . This modification of LSTM allows to use an unprojected hidden state while still communicating an output projected to the size of a velocity pair. The projected output and the cell state are communicated so that $\mathbf{r}_c = [\mathbf{c}_H^{\text{en}T} \quad \mathbf{h}_{\text{FFL},H}^{\text{en}T}]^T$. A decoder layer of this architecture is presented in Fig. 5. The decoder uses the communicated cell state as the initial cell state and the projected output as the initial input while the initial hidden states are set to zero.

A similar architecture has also been implemented to communicate only the output, i.e., $\mathbf{r}_c = \mathbf{h}_H^{\text{en}}$, in the following called LSTM+FFLH. The received output is used as the initial hidden state and additionally it is projected using an FFL to serve as the initial input. To act as the input of the next pass, the output of a forward pass is reduced using the same single-layer FNN as is used to reduce the received output. The cell state is initially set to zero. Note that, in multi-layer LSTMs, only the cell or network output of the last layer is communicated and consequently also only the respective states of the first layer in the decoder are set to a non-zero vector.

4 Data Generation, Tuning, and Training

Three main steps are taken to employ a learning-based communication strategy for DMPC, namely data generation by recording inter-agent communication, training and tuning of the encoder-decoder architecture as a standalone model, and, finally, splitting the model into its components and implementing the encoder prior to the transmission and the decoder post reception. The dataset consists of the recorded candidate control inputs $\mathbf{u}^{[p]}(\cdot|t)$ for $p \in \mathbb{Z}_{0:\bar{p}}$ for each $t \in \mathbb{Z}_{0:N_{\text{end}}}$ published by a single agent with the number of time steps $N_{\text{end}} := \lceil \Delta t^{-1} T_{\text{end}} \rceil$ where T_{end} is the simu-

lation duration. During training, the control logic of each robot runs in a separate process but all processes run on a shared compute unit. For the data generation, only formations consisting of two robots have been used. Using only the communicated predictions of one robot from two-robot formations is sufficient as in simulations without disturbances, all robots in a formation make similar predictions and cover the input space similarly well. The initial pose of the geometric center is uniformly randomly chosen from $x, y \in [-10 \text{ m}, 10 \text{ m}]$ and $\theta \in [-\pi \text{ rad}, \pi \text{ rad}]$. The robots are placed with $x_{\text{rel}} = \pm 0.5 \text{ m}$ relative to the geometric center using the same y -position and orientation as the geometric center. As, for control, only the relative positioning (i.e., the error) toward the goal formation-pose matters, it is enough to vary either the starting or the goal poses, where we opt for the former. The desired geometric center is always set to $\hat{\mathbf{y}}_d = [0 \text{ m} \quad 0 \text{ m} \quad 0 \text{ rad}]^T$.

For horizon length $H = 20$, 2000 randomly generated scenarios are used in the training set. Data generated from these scenarios is used to train the neural network architectures with fixed length. To train encoder-decoder LSTMs for variable horizon lengths, data is additionally recorded for 200 scenarios for each horizon length $H \in \mathbb{Z}_{21:25}$. The 200 scenarios are a random subset of the 2000 scenarios used for $H = 20$.

The simulation duration for the data generation is set to $T_{\text{end}} = 50 \text{ s}$. This gives the formations enough time to approach the setpoint and ensures that robots do not spend too much time in close proximity to the desired pose. For the data generation, training, and when testing the performance of the communication reduction, the following setup is used if not stated otherwise. The distributed controller is implemented in C++ using CasADi [33] to perform automatic differentiation and Eigen [34] for linear algebra. Robots communicate via UDP multicast using ZCM [35], which is based on LCM [36]. The initial step size $\bar{\alpha} = 0.1$, the backtracking factor $\beta = 0.5$, and the Armijo factor $\sigma = 0.5$ are used based on previous experience with the employed distributed optimization algorithm [20]. The number of iterations per time step is set to $\bar{p} = 3$ and the sampling time is $\Delta t = 0.25 \text{ s}$. The first candidate inputs of the optimization at the end of each time step, i.e., $\mathbf{u}^{[p]}(t|t)$, are communicated to a simulator program written in Python, which updates the robots' poses and sends the updated poses back. The simulator uses a multi-body model, previously used in [22, 28, 37] including actuation dynamics and inertia of the chassis and the wheels, and hence having been found to translate well to real-world experiments. This introduces a model mismatch between prediction and simulation of the robot motions.

For each neural network architecture, hyperparameter tuning is performed. The models are implemented in PyTorch [38] and training runs are tracked using Weights & Biases [39]. The dataset is split into 90% training data and 10% validation data. Employing the encoder-decoder structure in the optimization algorithm and running formation tasks is considered the test phase, see Section 5. The initial learning rate is set to either 0.001 or 0.01 combined with a reduce-on-plateau learning rate scheduler, reducing the learning rate by a factor of 0.1 if the validation loss

Table 2: Hyperparameter settings with lowest mean validation loss over an epoch for the considered architectures.

	SHRED	Cell LSTM	LSTMP	LSTM+FFL	LSTM+FFLH
LSTM layer	2	2	2	2	2
Activation function	tanh			tanh/relu	tanh
Inner layer	25				
Outer layer	35				
batch size	1024	256	256	256	256
Validation Loss	0.00039	0.00093	0.00130	0.000574/0.000568	0.00059

does not decrease for five epochs. With this approach, the learning has mostly ceased after 200 epochs, such that 200 is chosen as an upper epoch limit for all tuning runs. For training, the batch sizes 265, 512, 1024 are considered. The training uses the ADAM optimizer [40] and the mean-square error loss function. In all neural network model setups, it is ensured that the code size, i.e., the dimension of the latent space, is 10 corresponding to the code size used for the autoencoder in the prior work [28], to have an ample baseline. This means that, in cases where only one of the states is communicated, the hidden and cell state size is 10. This is the case for SHRED, cell LSTM, and LSTM+FFLH. In other cases, where a cell state and the network output are communicated together, the hidden and cell state size is set to eight and the output size is two, as it represents an extended term of the prediction. This applies to LSTMP and LSTM+FFL. LSTMs are considered with one to three layers. For the LSTM+FFL architecture, a single FNN layer is considered with input size eight and output size two. As activation functions, hyperbolic tangent and ReLU are considered. The FNN layer in the LSTM+FFLH has an input size of 10 and an output size of two. The FNN decoder of SHRED is tuned using the hyperbolic tangent, ReLU, and leaky ReLU activation functions. The decoder has two hidden layers. The inner layer, i.e., the hidden layer closer to the code, has one of the sizes 25, 23, 20, 18, or 15. The outer layer, i.e., the hidden layer closer to the output, is either 35, 33, 30, 28, or 26. The small layer sizes are chosen to ensure a low computational overhead.

The best parameter setup found for each variant are summarized in Table 2. For the LSTM+FFL variant, the ReLU activation function worked slightly better in training but using the hyperbolic tangent outperformed the ReLU activation function in testing. The validation loss row shows that the SHRED-based variant yielded the best whereas the LSTMP variant yielded the worst training performance. When considering only architectures trained for a variety of prediction horizons, the LSTM+FFL combination seems to train best.

5 Test: Simulative Analysis

This section focuses on testing the convergence performance of the optimization algorithm when incorporating the trained models and reducing the communication under idealized communication conditions. This corresponds to the test phase of the model training. For the analysis in this section, all processes run on a single computer and no commu-

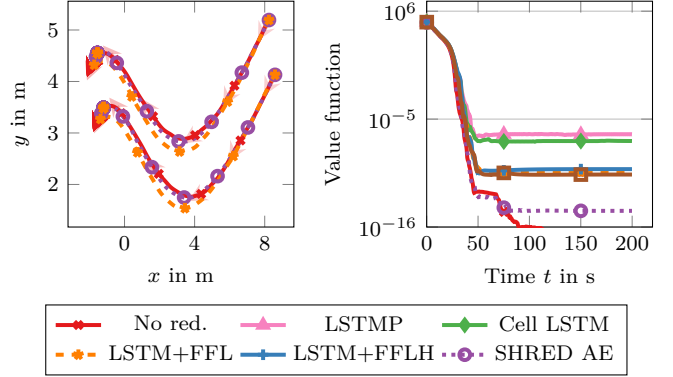


Figure 6: Left: Trajectories of an example scenario with two robots for $H = 20$ using no reduction (No red.) in the communication, SHRED, and LSTM+FFL communication reduction. The arrows in the left plot indicate the robot orientations for the task with full communication with the opacity increasing with time. Right: The corresponding value functions including all tested communication architectures for one robot.

nication between physical devices takes place. No packets are lost in this idealized setup. Afterwards, in the subsequent section, the focus shifts to determining how different communication approaches perform under demanding and realistic wireless communication conditions.

To quantify the performance, 200 scenarios for formation sizes from two to six robots are considered for $H = 20$. The simulation duration $T_{\text{end}} = 200$ s is used for formations of two to four robots while larger formations require more time to allow testing convergence empirically. Thus, a simulation duration of $T_{\text{end}} = 300$ s and $T_{\text{end}} = 450$ s is used for formations of five and six robots respectively. Initial and desired pose of the geometric center are chosen randomly with uniform distribution for each scenario within the regions $x, y \in [-20 \text{ m}, 20 \text{ m}]$ for the position and $\theta \in [-\pi \text{ rad}, \pi \text{ rad}]$ for the orientation. A test scenario consisting of a formation with two robots is exemplarily displayed in Fig. 6. As an additional baseline, the retrained autoencoder from [28] is included. The summary of the average performances of the different communication techniques is presented in Fig. 7. For $H = 20$, the horizon length contributing the most samples to the training dataset, SHRED outperforms the other techniques. Of the variants compatible with and trained for a variable horizon length, the combination of LSTM and FNN performed better than cell LSTM and LSTMP. This matches the training results. Looking at the best and worst

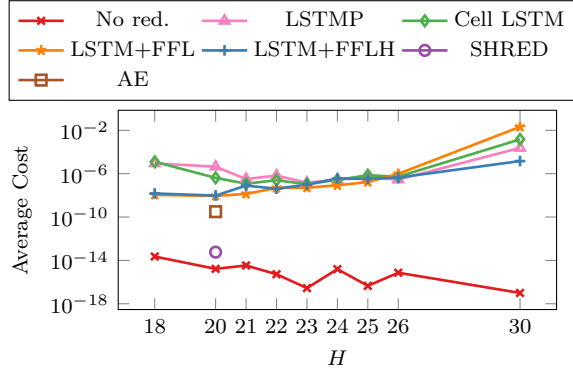


Figure 7: Mean cost values at T_{end} of one robot for the test scenarios for all communication reduction techniques. The horizon lengths are connected with lines to improve readability.

Table 3: Overview of cost at T_{end} for $H = 20$.

	Avg.	Max.	Min.
No reduction	$1.66 \cdot 10^{-15}$	$9.38 \cdot 10^{-14}$	$< 10^{-16}$
AE	$3.12 \cdot 10^{-10}$	$9.89 \cdot 10^{-9}$	$2.03 \cdot 10^{-11}$
SHRED	$5.76 \cdot 10^{-14}$	$2.82 \cdot 10^{-13}$	$4.20 \cdot 10^{-15}$
Cell LSTM	$4.14 \cdot 10^{-7}$	$7.29 \cdot 10^{-6}$	$5.33 \cdot 10^{-8}$
LSTMP	$4.41 \cdot 10^{-6}$	$2.13 \cdot 10^{-5}$	$2.69 \cdot 10^{-7}$
LSTM+FFL	$8.89 \cdot 10^{-9}$	$4.90 \cdot 10^{-7}$	$2.99 \cdot 10^{-11}$
LSTM+FFLH	$9.66 \cdot 10^{-9}$	$6.74 \cdot 10^{-7}$	$6.01 \cdot 10^{-11}$

performance confirms this observation, see Table 3. That methods trained for variable horizon lengths perform worse on a specific horizon than methods specifically trained for this horizon length is expected as they sacrifice some accuracy for generality. The findings for $H = 20$ are further supported by the value function plot on the right of Fig. 6 for one example scenario. The value function corresponds to the cost function values at the last iterate of each time step, i.e., the (sub-)optimal solution. An important performance measure specifically for the considered nonholonomic mobile robots is the error in the hard-to-control lateral-direction, which is the y direction at the setpoint. To measure the per-

formance when using different communication techniques, the cumulative density functions (CDFs) of the y -error at $T_{\text{end}} = 300$ s of 60 different scenarios consisting of formations of two to four robots are compared. The CDFs are depicted in Fig. 8. The formations depicted in the CDF plot cover a smaller area than the scenarios used for the cost comparison. The CDFs confirm the previous observations, full communication achieves the lowest y -error with 100% reaching a deviation of less than $1.0 \cdot 10^{-9}$ m, closely followed by SHRED. SHRED’s y -error is almost two orders of magnitude lower than the autoencoder error. The combinations of LSTM and FNN come close to the performance of the autoencoder towards 100% while the other LSTM variants barely reach an accuracy below 1 mm.

For each of the horizon lengths $H \in \{18, 21, 22, 23, 24, 25, 26, 30\}$, the same tasks are used for testing. Looking at the average cost at T_{end} for different prediction horizons included in the training scenarios in Table 4, the architectures combining an LSTM element with an FFL projection have the lowest average cost at T_{end} in most of the horizon length covered by the training. The same behavior can be observed for $H = 18$, a horizon length outside of the training range. However, for increasingly longer horizons, the cell LSTM and LSTMP variant begin to have lower cost values at T_{end} compared to the LSTM+FFL variant. For $H = 30$, well outside the prediction horizon training range, the LSTM+FFL variant performs worst, but the LSTM+FFLH variant retains best performance. The CDFs for the y -deviation under equal conditions as used for $H = 20$ show that LSTMP and cell LSTM perform better for longer prediction horizons, even slightly outside the training range. At $H = 26$, all architectures experience similar convergence behavior. The LSTM+FFLH model performs similarly for all tested horizon lengths with 100% of scenarios reaching a y -deviation of about 10^{-5} m, even at $H = 30$ where the other models perform significantly worse and where the LSTM+FFL model does not achieve an accuracy smaller than 1 mm for any of the considered scenarios.

6 Numerical Experiments on Embedded Hardware

To compare full communication to semantically reduced communication in a realistic communication setting, the distributed optimization algorithm is run on Raspberry Pi 5 single-board computers with 8 GB RAM, as they are also fitted to the robots depicted in Fig. 1. The optimization algorithm for one robot is run on a dedicated Raspberry Pi. The communication takes place via 5 GHz Wi-Fi using UDP multicast. Each robot re-publishes messages once if it has not received all messages it is waiting for after half of the time dedicated to an iteration. Additionally, robots use the best information available, i.e., if no new message is received, old information is reused and if information of a subsequent iteration but not of the current iteration is available, the information meant for the next iteration is already used. The experimental setup is exemplarily depicted in Fig. 10. The

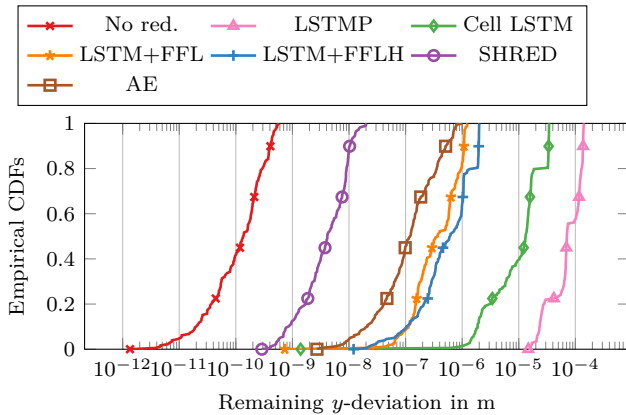


Figure 8: Empirical cumulative distribution functions (CDFs) of error in y -direction for 60 random formation tasks with full and reduced communication.

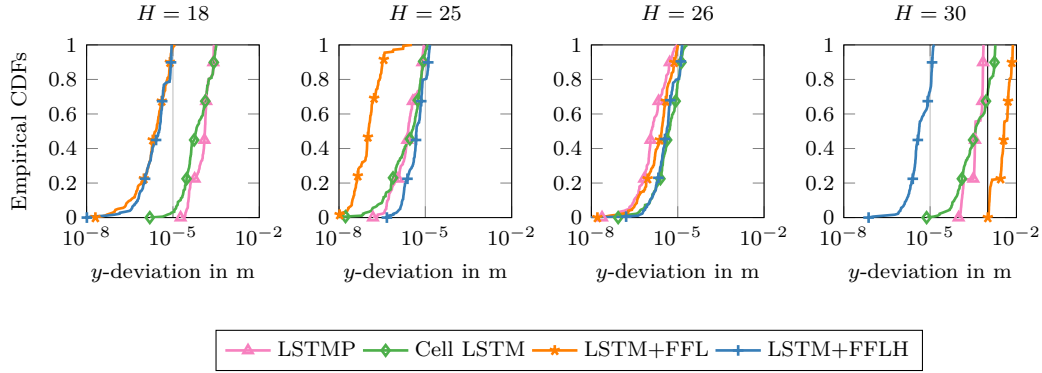


Figure 9: Empirical cumulative distribution functions (CDFs) of error in y -direction for 60 random formation tasks with encoder-decoder LSTM-reduced communication for different H values.

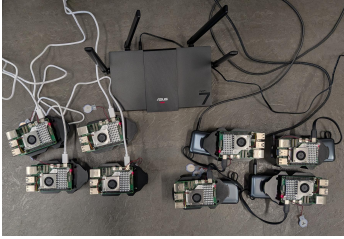


Figure 10: Setup for the numerical experiments

numerical experiment only considers simulated mechanical dynamics. This simplifies the experiment in two ways as the challenges of moving communication endpoints are neglected and so are deviations between prediction and the behavior of the real robot beyond model mismatches. The simulator program runs on a personal computer with an Intel Core Ultra 5 238V @ 2.1 GHz and 32 GB of memory, which is connected to the router via LAN. It receives the applied inputs from each robot and computes the new poses and publishes the updated poses. This introduces additional communication but moves the application of the control input from the on-board computer to an external computer thus slightly modifying the setup. As reduction methods only SHRED and LSTM+FFLH are considered as the best performing methods from Section 5. The choice of LSTM+FFLH over LSTM+FFL is motivated by the better generalization to larger values of the prediction horizon lengths. Parallel parking tasks as depicted exemplarily for four robots in Fig. 11 are used in the numerical experiments.

Each experiment (except for the case with $\bar{p} = 3$, as under these conditions all variants work well) was repeated a few times to account for a varying experimental environment that could be impacting the performance. In the following, the best performing repetition is depicted in value-function plots.

In the first experiment, a formation of eight robots is chosen combined with $H = 20$ and $\bar{p} = 3$ aligning with the setup chosen for the simulative analysis. Under these conditions, all three communication configurations manage to receive around 99% of the amount of communicated information,

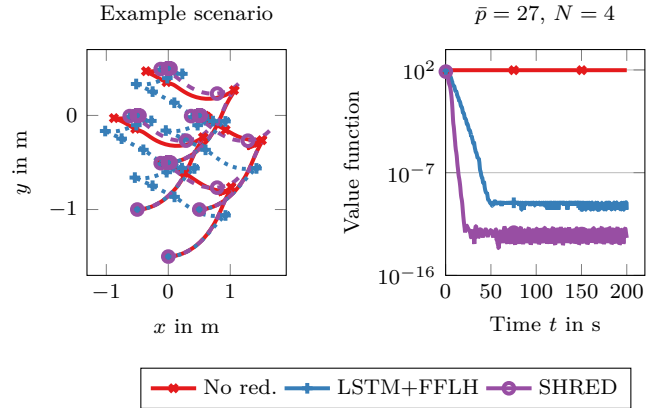


Figure 11: Left: Trajectories of an example parallel parking scenario with four robots using no reduction (No red.) in the communication, SHRED, and LSTM+FFLH communication reduction. Right: The value functions of one of the robots of a scenario with four robots.

see Table 5, and display convergence similar to the simulation results, as shown in Fig. 12. Using a smaller formation of four robots and increasing the number of iterations to $\bar{p} = 27$, with full communication, none of the inter-agent messages arrive in time while reduced communication still converge to small value-function values, see Fig. 11. However, LSTM+FFLH receives around 8 percentage points less information on average compared to SHRED (Table 5) although both communicate the same amount of information and use identical packet sizes. A similar result can be observed when increasing the number of agents to eight. Here, the full communication already collapses at $\bar{p} = 12$ due to more demanding communication conditions while SHRED continues to converge to small value function values, see both Table 5 and Fig. 12, whereas LSTM+FFLH receives less information than in the scenario with four robots and fails to converge. This performance decline is likely due to the computational overhead of the decoder LSTM, requiring about six times the amount of time compared to the FNN decoder used for SHRED per robot leading also to a decline in received state updates under challenging conditions. Increasing H to 25, which SHRED cannot handle with-

Table 4: Mean, max, and min cost values at T_{end} of one robot for the test scenarios for all communication reduction techniques working for a variable horizon length. For each horizon length, the best performing reduction method is highlighted.

	H	Avg.	Max	Min
No reduction	18	$2.29 \cdot 10^{-14}$	$2.26 \cdot 10^{-12}$	$< 10^{-16}$
Cell LSTM	18	$1.24 \cdot 10^{-5}$	$5.25 \cdot 10^{-5}$	$1.57 \cdot 10^{-6}$
LSTMP	18	$8.92 \cdot 10^{-6}$	$3.054 \cdot 10^{-5}$	$6.45 \cdot 10^{-7}$
LSTM+FFL	18	$1.11 \cdot 10^{-8}$	$6.55 \cdot 10^{-8}$	$4.28 \cdot 10^{-10}$
LSTM+FFLH	18	$1.50 \cdot 10^{-8}$	$2.34 \cdot 10^{-7}$	$9.81 \cdot 10^{-10}$
No reduction	21	$3.42 \cdot 10^{-15}$	$3.08 \cdot 10^{-13}$	$< 10^{-16}$
Cell LSTM	21	$1.20 \cdot 10^{-7}$	$2.26 \cdot 10^{-6}$	$1.09 \cdot 10^{-8}$
LSTMP	21	$3.20 \cdot 10^{-7}$	$5.11 \cdot 10^{-6}$	$2.99 \cdot 10^{-8}$
LSTM+FFL	21	$1.36 \cdot 10^{-8}$	$7.06 \cdot 10^{-7}$	$8.22 \cdot 10^{-11}$
LSTM+FFLH	21	$7.84 \cdot 10^{-8}$	$5.89 \cdot 10^{-6}$	$7.09 \cdot 10^{-11}$
No reduction	22	$5.27 \cdot 10^{-16}$	$3.02 \cdot 10^{-14}$	$< 10^{-16}$
Cell LSTM	22	$2.48 \cdot 10^{-7}$	$1.08 \cdot 10^{-5}$	$1.40 \cdot 10^{-8}$
LSTMP	22	$6.68 \cdot 10^{-7}$	$3.59 \cdot 10^{-6}$	$3.69 \cdot 10^{-8}$
LSTM+FFL	22	$4.82 \cdot 10^{-8}$	$3.11 \cdot 10^{-6}$	$7.39 \cdot 10^{-11}$
LSTM+FFLH	22	$4.13 \cdot 10^{-8}$	$2.08 \cdot 10^{-6}$	$4.62 \cdot 10^{-11}$
No reduction	23	$< 10^{-16}$	$1.50 \cdot 10^{-15}$	$< 10^{-16}$
Cell LSTM	23	$1.08 \cdot 10^{-7}$	$4.84 \cdot 10^{-6}$	$2.22 \cdot 10^{-9}$
LSTMP	23	$1.47 \cdot 10^{-7}$	$2.77 \cdot 10^{-6}$	$1.76 \cdot 10^{-8}$
LSTM+FFL	23	$5.10 \cdot 10^{-8}$	$2.57 \cdot 10^{-6}$	$5.44 \cdot 10^{-11}$
LSTM+FFLH	23	$9.66 \cdot 10^{-8}$	$4.88 \cdot 10^{-6}$	$2.37 \cdot 10^{-10}$
No reduction	24	$1.57 \cdot 10^{-15}$	$1.24 \cdot 10^{-13}$	$< 10^{-16}$
Cell LSTM	24	$2.72 \cdot 10^{-7}$	$1.29 \cdot 10^{-5}$	$3.94 \cdot 10^{-9}$
LSTMP	24	$2.83 \cdot 10^{-7}$	$1.52 \cdot 10^{-5}$	$1.01 \cdot 10^{-8}$
LSTM+FFL	24	$8.43 \cdot 10^{-8}$	$5.59 \cdot 10^{-6}$	$2.24 \cdot 10^{-10}$
LSTM+FFLH	24	$3.61 \cdot 10^{-7}$	$2.72 \cdot 10^{-5}$	$1.40 \cdot 10^{-9}$
No reduction	25	$< 10^{-16}$	$3.10 \cdot 10^{-15}$	$< 10^{-16}$
Cell LSTM	25	$7.16 \cdot 10^{-7}$	$6.14 \cdot 10^{-5}$	$5.27 \cdot 10^{-9}$
LSTMP	25	$4.61 \cdot 10^{-7}$	$3.53 \cdot 10^{-5}$	$9.75 \cdot 10^{-9}$
LSTM+FFL	25	$1.69 \cdot 10^{-7}$	$9.95 \cdot 10^{-6}$	$3.16 \cdot 10^{-12}$
LSTM+FFLH	25	$3.30 \cdot 10^{-7}$	$2.13 \cdot 10^{-5}$	$3.05 \cdot 10^{-9}$
No reduction	26	$7.30 \cdot 10^{-16}$	$5.05 \cdot 10^{-14}$	$< 10^{-16}$
Cell LSTM	26	$5.31 \cdot 10^{-7}$	$3.07 \cdot 10^{-5}$	$2.09 \cdot 10^{-8}$
LSTMP	26	$2.86 \cdot 10^{-7}$	$2.01 \cdot 10^{-5}$	$9.02 \cdot 10^{-9}$
LSTM+FFL	26	$9.66 \cdot 10^{-7}$	$1.50 \cdot 10^{-5}$	$1.90 \cdot 10^{-7}$
LSTM+FFLH	26	$4.39 \cdot 10^{-7}$	$3.13 \cdot 10^{-5}$	$2.55 \cdot 10^{-9}$
No reduction	30	$< 10^{-16}$	$6 \cdot 10^{-16}$	$< 10^{-16}$
Cell LSTM	30	0.00146	0.00457	0.00038
LSTMP	30	0.000238	0.00066	$3.40 \cdot 10^{-5}$
LSTM+FFL	30	0.02069	0.05110	0.00380
LSTM+FFLH	30	$1.45 \cdot 10^{-5}$	0.00014	$2.92 \cdot 10^{-6}$

out retraining, both communication without reduction and LSTM+FFLH-based reduction struggle already at $\bar{p} = 10$. Although LSTM+FFLH received over 90% of information, it struggles to converge while, interestingly, without communication reduction, the formation managed to converge sometimes even when receiving less than 40% of the communicated information. Under these conditions, LSTM+FFLH receives substantially less of the state updates in time. This is likely due to the increased computational overhead of the decoder resulting in the optimization using some of the time reserved for state updates.

7 Conclusion

This work proposed and analyzed different neural network architectures to construct semantic encoding for data trans-

Table 5: Average amount of communicated information received in percent on embedded hardware

Name	N	T_{end}	H	$\bar{p}$	Avg. info. received in %
No red.	8	800	20	3	98.97
SHRED	8	800	20	3	99.02
LSTM+FFLH	8	800	20	3	99.12
No red.	4	200	20	27	0.00
SHRED	4	200	20	27	98.29
LSTM+FFLH	4	200	20	27	90.70
No red.	8	200	20	12	0.02
SHRED	8	200	20	12	98.01
LSTM+FFLH	8	200	20	12	87.43
No red.	8	300	25	10	31.41
LSTM+FFLH	8	300	25	10	93.38

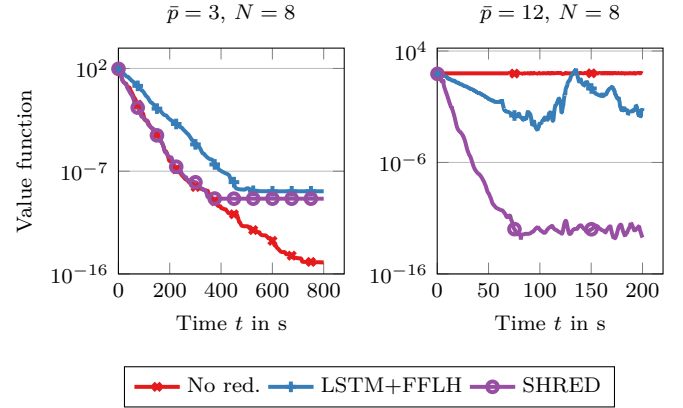


Figure 12: Value functions for two experiments with $N = 8$ and $H = 20$

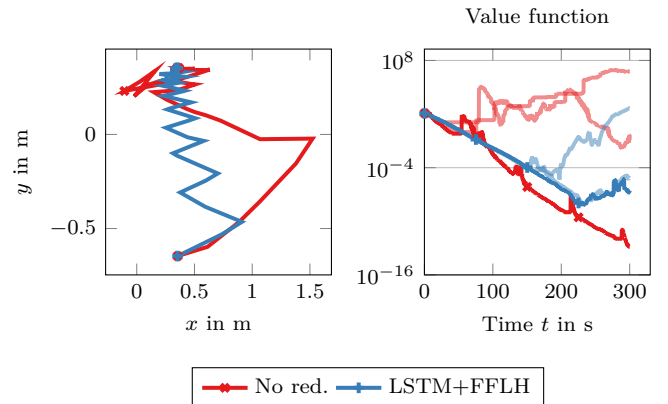


Figure 13: Left: Trajectories for one robot of an example scenario with $H = 25$ and $\bar{p} = 10$ using no reduction (No red.) in the communication and LSTM+FFLH communication reduction. Right: The value functions of one of the robots for all conducted experiment repetitions.

mission based on LSTMs to reduce the inter-agent communication in distributed model predictive control. Using SHRED to reduce the inter-agent communication resulted in convergence performance closely matching the performance of full communication in simulation and has been shown to perform well in experiments with physical wireless communication even in conditions where full communication faltered. Additionally, SHRED outperforms the autoencoder-based communication reduction from [28]. However, SHRED uses a standard FNN in the decoder paired with an LSTM encoder, such that retraining is required as soon as the prediction horizon length is changed. Although the training process may be automated as it proved benign, it introduces a burden.

Using an LSTM also in the decoder allows to train a single model for multiple prediction horizons. This generalization affected specialization such that none of the models managed to match the performances of SHRED. Moreover, an encoder-decoder LSTM also manages to converge under some conditions where the communication without reduction is overwhelmed. However, the larger the formation or the longer the prediction horizon, the larger the computational overhead is in the decoder counteracting the benefits of the communication reduction and shifting the bottleneck from communication back to computation. In these experiments, it was seen that DMPC can be surprisingly robust, sometimes converging when only receiving about a third of the amount of communicated information. Whether and when there is inherent robustness to lossy or intermittent communication in DMPC could be worth further investigation from a theoretical standpoint, compare the recent work [41] investigating issues like this for a different setup. In a next step, the methods should be tested in a setup including the additional challenge that moving communication endpoints introduce. Additionally, the internal structure of the OCP solutions along the prediction horizon may be considered more specifically to guide training.

8 Acknowledgments

This work was partly supported by (a) the Research Council of Finland through (i) ECO-NEWS (No. 358928), (ii) XSDEN (No. 349965), and (iii) OptAd (No. 372723); (b) EU MSCA COALESCE project (No. 101130739).

References

- [1] Samad, T.; Bauer, M.; Bortoff, S.; Di Cairano, S.; Fagiano, L.; Odgaard, P.F.; Rhinehart, R.R.; Sánchez-Peña, R.; Serbezov, A.; Ankersen, F.; Goupil, P.; Grosman, B.; Heertjes, M.; Mareels, I.; Sosseh, R.: Industry engagement with control research: perspective and messages. *Annual Reviews in Control*, Vol. 49, pp. 1–14, 2020. DOI: [10.1016/j.arcontrol.2020.03.002](https://doi.org/10.1016/j.arcontrol.2020.03.002)
- [2] Negenborn, R.R.; Maestre, J.M.: Distributed model predictive control: an overview and roadmap of future research opportunities. *IEEE Control Systems*, Vol. 34, No. 4, pp. 87–97, 2014. DOI: [10.1109/MCS.2014.2320397](https://doi.org/10.1109/MCS.2014.2320397)
- [3] Rawlings, J.B.; Mayne, D.Q.; Diehl, M.: *Model predictive control: theory, computation and design*. Santa Barbara: Nob Hill Publishing, LLC, 2nd edition. Edn., 2020.
- [4] Stewart, B.T.; Wright, S.J.; Rawlings, J.B.: Cooperative distributed model predictive control for nonlinear systems. *Journal of Process Control*, Vol. 21, No. 5, pp. 698–704, 2011. DOI: [10.1016/j.jprocont.2010.11.004](https://doi.org/10.1016/j.jprocont.2010.11.004)
- [5] Stomberg, G.; Engelmann, A.; Diehl, M.; Faulwasser, T.: Decentralized real-time iterations for distributed NMPC. *IEEE Transactions on Automatic Control*, pp. 1–16, 2025. DOI: [10.1109/TAC.2025.3622000](https://doi.org/10.1109/TAC.2025.3622000)
- [6] Müller, M.A.; Reble, M.; Allgöwer, F.: Cooperative control of dynamically decoupled systems via distributed model predictive control. *International Journal of Robust and Nonlinear Control*, Vol. 22, No. 12, pp. 1376–1397, 2012. DOI: [10.1002/rnc.2826](https://doi.org/10.1002/rnc.2826)
- [7] Köhler, J.; Müller, M.A.; Allgöwer, F.: Distributed model predictive control—recursive feasibility under inexact dual optimization. *Automatica*, Vol. 102, pp. 1–9, 2019. DOI: [10.1016/j.automatica.2018.12.037](https://doi.org/10.1016/j.automatica.2018.12.037)
- [8] Van Parys, R.; Pipeleers, G.: Distributed MPC for multi-vehicle systems moving in formation. *Robotics and Autonomous Systems*, Vol. 97, pp. 144–152, 2017. DOI: [10.1016/j.robot.2017.08.009](https://doi.org/10.1016/j.robot.2017.08.009)
- [9] Stomberg, G.; Schwan, R.; Grillo, A.; Jones, C.N.; Faulwasser, T.: Cooperative distributed model predictive control for embedded systems: experiments with hovercraft formations. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11377–11383, Atlanta, GA, USA: IEEE, 2025. DOI: [10.1109/ICRA55743.2025.11128032](https://doi.org/10.1109/ICRA55743.2025.11128032)
- [10] Pappas, N.; Kountouris, M.; Niu, X.; Ephremides, A.: Goal-oriented semantic communication and networking. *IEEE Network*, Vol. 38, No. 6, 2024. DOI: [10.1109/MNET.2024.3466368](https://doi.org/10.1109/MNET.2024.3466368)
- [11] Fountoulakis, E.; Pappas, N.; Kountouris, M.: Goal-oriented policies for cost of actuation error minimization in wireless autonomous systems. *IEEE Communications Letters*, Vol. 27, No. 9, pp. 2323–2327, 2023. DOI: [10.1109/LCOMM.2023.3288070](https://doi.org/10.1109/LCOMM.2023.3288070)
- [12] Chaccour, C.; Saad, W.; Debbah, M.; Han, Z.; Poor, H.V.: Less data, more knowledge: building next-generation semantic communication networks. *IEEE Communications Surveys & Tutorials*, Vol. 27, No. 1, pp. 37–76, 2025. DOI: [10.1109/COMST.2024.3412852](https://doi.org/10.1109/COMST.2024.3412852)

- [13] Silva, P.E.G.; Nardelli, P.H.J.; de Sena, A.S.; Siljak, H.; Nevaranta, N.; Marchetti, N.: Semantic-functional communications in cyber-physical systems. *IEEE Network*, Vol. 38, No. 4, pp. 241–249, 2024. DOI: [10.1109/MNET.2023.3329192](https://doi.org/10.1109/MNET.2023.3329192)
- [14] Uysal, E.; Kaya, O.; Ephremides, A.; Gross, J.; Codreanu, M.; Popovski, P.; Assaad, M.; Liva, G.; Munari, A.; Soret, B.; Soleymani, T.; Johansson, K.H.: Semantic communications in networked systems: a data significance perspective. *IEEE Network*, Vol. 36, No. 4, pp. 233–240, 2022. DOI: [10.1109/MNET.106.2100636](https://doi.org/10.1109/MNET.106.2100636)
- [15] Charalambous, T.; Pappas, N.; Nomikos, N.; Wichman, R.: Toward goal-oriented communication in multi-agent systems: An Overview. *IEEE Open Journal of the Communications Society*, Vol. 7, pp. 6576–6611, 2026. DOI: [10.1109/OJCOMS.2026.3703748](https://doi.org/10.1109/OJCOMS.2026.3703748)
- [16] Cho, K.; Van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; Bengio, Y.: Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, Doha, Qatar: Association for Computational Linguistics, 2014. DOI: [10.3115/v1/D14-1179](https://doi.org/10.3115/v1/D14-1179)
- [17] Fuchs, M.; Leprich, D.; Rosenfelder, M.; Eberhard, P.: Distributed cooperative transportation of an elastic object using omnidirectional mobile robots. *Multibody System Dynamics*, 2026. DOI: [10.1007/s11044-026-10181-6](https://doi.org/10.1007/s11044-026-10181-6)
- [18] Ebel, H.: Distributed control and organization of communicating mobile robots: design, simulation, and experimentation, Vol. 69 of Dissertation, Schriften aus dem Institut für Technische und Numerische Mechanik der Universität Stuttgart. Düren: Shaker Verlag, 2021. DOI: [10.2370/9783844081725](https://doi.org/10.2370/9783844081725)
- [19] Burk, D.; Volz, A.; Graichen, K.: Experimental validation of the open-source DMPC framework GRAMPC-D applied to the remotely accessible robotarium. In *2021 IEEE International Conference on Mechatronics and Automation (ICMA)*, pp. 442–447, Takamatsu, Japan: IEEE, 2021. DOI: [10.1109/ICMA52036.2021.9512773](https://doi.org/10.1109/ICMA52036.2021.9512773)
- [20] Rosenfelder, M.; Ebel, H.; Eberhard, P.: Cooperative distributed nonlinear model predictive control of a formation of differentially-driven mobile robots. *Robotics and Autonomous Systems*, Vol. 150, p. 103993, 2022. DOI: [10.1016/j.robot.2021.103993](https://doi.org/10.1016/j.robot.2021.103993)
- [21] Stomberg, G.; Ebel, H.; Faulwasser, T.; Eberhard, P.: Cooperative distributed MPC via decentralized real-time optimization: implementation results for robot formations. *Control Engineering Practice*, Vol. 138, p. 105579, 2023. DOI: [10.1016/j.conengprac.2023.105579](https://doi.org/10.1016/j.conengprac.2023.105579)
- [22] Ebel, H.; Rosenfelder, M.; Eberhard, P.: Cooperative object transportation with differential-drive mobile robots: control and experimentation. *Robotics and Autonomous Systems*, Vol. 173, p. 104612, 2024. DOI: [10.1016/j.robot.2023.104612](https://doi.org/10.1016/j.robot.2023.104612)
- [23] Gräfe, A.; Eickhoff, J.; Zimmerling, M.; Trimpe, S.: DMPC-swarm: distributed model predictive control on nano UAV swarms. *Autonomous Robots*, Vol. 49, No. 4, p. 28, 2025. DOI: [10.1007/s10514-025-10211-w](https://doi.org/10.1007/s10514-025-10211-w)
- [24] Luo, X.; Chen, H.H.; Guo, Q.: Semantic communications: overview, open issues, and future research directions. *IEEE Wireless Communications*, Vol. 29, No. 1, pp. 210–219, 2022. DOI: [10.1109/MWC.101.2100269](https://doi.org/10.1109/MWC.101.2100269)
- [25] Wengle, E.; Varagnolo, D.: Distributed MPC formation path following for acoustically communicating underwater vehicles, 2024. DOI: [10.48550/arXiv.2410.11959](https://doi.org/10.48550/arXiv.2410.11959)
- [26] DiLeo, N.; Abad, A.; Pregene, K.: Communications aware decentralized model predictive control for path planning within UUV swarms. In *2017 IEEE Conference on Control Technology and Applications (CCTA)*, pp. 249–254, Mauna Lani Resort, HI, USA: IEEE, 2017. DOI: [10.1109/CCTA.2017.8062471](https://doi.org/10.1109/CCTA.2017.8062471)
- [27] El-Ferik, S.; Siddiqui, B.A.; Lewis, F.L.: Distributed nonlinear MPC of multi-agent systems with data compression and random delays. *IEEE Transactions on Automatic Control*, Vol. 61, No. 3, pp. 817–822, 2016. DOI: [10.1109/TAC.2015.2449791](https://doi.org/10.1109/TAC.2015.2449791)
- [28] Schiz, T.; Ebel, H.: Reducing the communication of distributed model predictive control: autoencoders and formation control. *Control Engineering Practice*, Vol. 165, p. 106560, 2025. DOI: [10.1016/j.conengprac.2025.106560](https://doi.org/10.1016/j.conengprac.2025.106560)
- [29] Worthmann, K.; Mehrez, M.W.; Zanon, M.; Mann, G.K.; Gosine, R.G.; Diehl, M.: Regulation of differential drive robots using continuous time MPC without stabilizing constraints or costs. *IFAC-PapersOnLine*, Vol. 48, No. 23, pp. 129–135, 2015. DOI: [10.1016/j.ifacol.2015.11.272](https://doi.org/10.1016/j.ifacol.2015.11.272)
- [30] Rosenfelder, M.; Ebel, H.; Eberhard, P.: Cooperative distributed model predictive formation control of non-holonomic robotic agents. In *2021 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*, pp. 11–19, Cambridge, United Kingdom: IEEE, 2021. DOI: [10.1109/MRS50823.2021.9620683](https://doi.org/10.1109/MRS50823.2021.9620683)
- [31] Williams, J.P.; Zahn, O.; Kutz, J.N.: Sensing with shallow recurrent decoder networks. *Proceedings of*

- the Royal Society A: Mathematical, Physical and Engineering Sciences, Vol. 480, No. 2298, p. 20240054, 2024. DOI: [10.1098/rspa.2024.0054](https://doi.org/10.1098/rspa.2024.0054)
- [32] Sak, H.; Senior, A.; Beaufays, F.: Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition, 2014. DOI: [10.48550/ARXIV.1402.1128](https://doi.org/10.48550/ARXIV.1402.1128)
- [33] Andersson, J.A.E.; Gillis, J.; Horn, G.; Rawlings, J.B.; Diehl, M.: CasADi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, Vol. 11, No. 1, pp. 1–36, 2019. DOI: [10.1007/s12532-018-0139-4](https://doi.org/10.1007/s12532-018-0139-4)
- [34] Guennebaud, G.; Jacob, B.; et al.: Eigen. <https://libeigen.gitlab.io>, 2010.
- [35] ZeroCM Contributors: Zero communications and marshallng (ZCM). <https://github.com/ZeroCM/zcm>. GitHub Repository, version 1.1.5.
- [36] Huang, A.S.; Olson, E.; Moore, D.C.: LCM: lightweight communications and marshallng. In 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 4057–4062, Taipei Taiwan: IEEE, 2010. DOI: [10.1109/IROS.2010.5649358](https://doi.org/10.1109/IROS.2010.5649358)
- [37] Ebel, H.; Fahse, D.N.; Rosenfelder, M.; Eberhard, P.: Finding formations for the non-prehensile object transportation with differentially-driven mobile robots. In Kecskeméthy, A.; Parenti-Castelli, V. (Eds.): ROMANSY 24 - Robot Design, Dynamics and Control, Vol. 606, pp. 163–170. Springer International Publishing, 2022. DOI: [10.1007/978-3-031-06409-8_17](https://doi.org/10.1007/978-3-031-06409-8_17)
- [38] Ansel, J.; Yang, E.; He, H.; Gimelshein, N.; Jain, A.; Voznesensky, M.; Bao, B.; Bell, P.; Berard, D.; Burovski, E.; Chauhan, G.; Chourdia, A.; Constable, W.; Desmaison, A.; DeVito, Z.; Ellison, E.; Feng, W.; Gong, J.; Gschwind, M.; Hirsh, B.; Huang, S.; Kalambarkar, K.; Kirsch, L.; Lazos, M.; Lezcano, M.; Liang, Y.; Liang, J.; Lu, Y.; Luk, C.K.; Maher, B.; Pan, Y.; Puhersch, C.; Reso, M.; Saroufim, M.; Siraichi, M.Y.; Suk, H.; Zhang, S.; Suo, M.; Tillet, P.; Zhao, X.; Wang, E.; Zhou, K.; Zou, R.; Wang, X.; Mathews, A.; Wen, W.; Chanan, G.; Wu, P.; Chintala, S.: PyTorch 2: faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, pp. 929–947, La Jolla CA USA: ACM, 2024. DOI: [10.1145/3620665.3640366](https://doi.org/10.1145/3620665.3640366)
- [39] Biewald, L.: Experiment tracking with weights and biases. <https://www.wandb.com/>, 2020. Software available from wandb.com. Accessed on 6 August 2026.
- [40] Kingma, D.P.; Ba, J.: Adam: a method for stochastic optimization, 2014. DOI: [10.48550/ARXIV.1412.6980](https://doi.org/10.48550/ARXIV.1412.6980)
- [41] Shen, S.; Zhang, C.; Lin, Z.; Chai, R.; Xia, Y.; Chai, S.: Inherent tolerance of distributed model predictive control to communication interruptions. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2026. DOI: [10.1109/TSMC.2026.3715932](https://doi.org/10.1109/TSMC.2026.3715932)